

AUIGridT.vue 서브 컴포넌트 설명서

AUIGridT.vue 는 Typescript + Vue.js 프레임워크에서 AUIGrid 를 쉽게 사용할 수 있도록 작성한 컴포넌트입니다.

1. 정의 가능한 속성들

속성명	유형(Type)	기본값	설명
autoResize	Boolean	true	resizeMode에 따른 자동 리사이징 사용 여부입니다. false이면 감지를 등록하지 않습니다.
columnLayout	IGrid.Column[]	[]	그리드 칼럼 레이아웃을 지정합니다.
footerLayout	IGrid.Footer[]	[]	그리드 푸터 레이아웃을 지정합니다.
gridProps	IGrid.Props	{}	그리드 속성을 지정합니다.
name	String	''	그리드 이름을 지정합니다.
resizeDelayTime	Number	300	마지막 크기 변경 알림 후 기다리는 시간(ms)입니다. 두 감지 방식에 모두 적용됩니다.
resizeMode	String	"window"	"window"는 창 크기 이벤트, "container"는 호스트 DIV의 크기 변화를 감지합니다. 4절을 참고하십시오.
waitPortalRendering	Boolean	false	Modal이나 Teleport 등 포탈 방식 렌더링을 위해 RAF 사용으로 생성할지 여부를 지정합니다.

* name 속성은 의무사항이 아닙니다. 그러나 name 을 지정하면 해당 name 값을 id 로 갖는 그리드가 생성됩니다. 예로 name='showcase01' 로 설정했다면 해당 그리드는 'aui-grid-wrap-showcase01' 을 id 로 갖는 그리드로 생성됩니다.

DOM 요소를 확인하면 다음과 같이 그리드 생성 div의 id 를 확인 할 수 있습니다.

```
<div id="aui-grid-wrap-showcase01" style="position: relative;"> == $0
  <div class="aui-grid" style="position: relative; box-sizing: content-box"
    <div>
```

단, 서버사이드 렌더링(SSR)을 하는 프로젝트나 AUIGrid.vue 가 동적으로 생성/제거되는 경우에 name은 고유값 설정이 의무화 됩니다.

2. 그리드에 접근하여 메소드 사용하기

Vue.js 에서 제공하는 ref 를 이용하여 다음처럼 참조하도록 합니다.

```
// 그리드 객체
const myGrid = ref<AUIGrid | null>(null);
```

위와 같이 ref 로 myGrid 를 생성한 후 AUIGrid 생성 태그에 ref 로 설정합니다

```
<AUIGrid ref="myGrid"/>
```

그러면 언제든지 Vue 자신의 스코프에서 다음처럼 그리드에 접근 할 수 있습니다.

```
const grid = myGrid.value as AUIGrid;
grid.addRow();
```

3. Vue.js에서 AUIGridT.vue 사용 예시

```
<script setup lang="ts">
  import { ref, onMounted } from 'vue';
  import * as IGrid from 'aui-grid';
  import AUIGrid from '@/static/AUIGrid-Vue/AUIGridT.vue';

  // 그리드 InstanceType
  type AUIGrid = InstanceType<typeof AUIGrid>;

  // 그리드 객체
  const myGrid = ref<AUIGrid | null>(null);

  // 그리드 속성 정의
  const gridProps: IGrid.Props = {
    width: '100%',
    height: 480,
    // 편집 가능 여부 (기본값 : false)
    editable: true,
    noDataMessage: '출력할 데이터가 없습니다.',
    groupingMessage: '여기에 칼럼을 드래그하면 그룹핑이 됩니다.'
  };

  // 그리드 칼럼 레이아웃 정의
  const columnLayout: IGrid.Column[] = [
    {
      dataField: 'id',
      headerText: 'ID',
      width: 120
    },
    {
```

```

        dataField: 'name',
        headerText: '이름',
        width: 140
    },
    {
        dataField: 'country',
        headerText: 'Country',
        width: 140
    },
    {
        dataField: 'product',
        headerText: 'Product',
        width: 140
    },
    {
        dataField: 'color',
        headerText: 'Color',
        width: 100
    },
    {
        dataField: 'price',
        headerText: 'Price',
        dataType: 'numeric',
        style: 'my-right-column',
        width: 120,
        editRenderer: {
            type: IGrid.EditRendererKind.InputEditRenderer,
            onlyNumeric: true, // 0~9 만 입력가능
            textAlign: 'right', // 오른쪽 정렬로 입력되도록 설정
            autoThousandSeparator: true // 천단위 구분자 삽입 여부
        }
    },
    {
        dataField: 'quantity',
        headerText: 'Quantity',
        dataType: 'numeric',
        style: 'my-right-column',
        width: 100,
        editRenderer: {
            type: IGrid.EditRendererKind.InputEditRenderer,
            onlyNumeric: true, // 0~9 만 입력가능
            textAlign: 'right', // 오른쪽 정렬로 입력되도록 설정
            autoThousandSeparator: true // 천단위 구분자 삽입 여부
        }
    },
    {
        dataField: 'date',

```

```

        headerText: 'Date',
        dataType: 'date',
        dateInputFormat: 'yyyy-mm-dd', // 데이터의 날짜 형식
        formatString: 'yyyy 년 mm 월 dd 일' // 그리드에 보여줄 날짜 형식
    }
];

const cellClick = (event: IGrid.EventKind.CellClick) => {
    console.log(event);
};

const requestGridData = async () => {
    const grid = myGrid.value;
    grid?.showAjaxLoader();
    const response = await fetch(BASE_URL + '/data/normal_100.json');
    const jsonData = await response.json();
    console.log(jsonData);
    grid?.setGridData(jsonData);
    grid?.removeAjaxLoader();
};

onMounted(() => {
    requestGridData();
});
</script>

<template>
    <div>
        <AUIGrid ref="myGrid" :gridProps="gridProps"
            :columnLayout="columnLayout" @cellClick="cellClick"/>
    </div>
</template>

```

4 컨테이너 크기 감지 사용하기

resizeMode="container"를 지정하면 ResizeObserver로 그리드를 담는 호스트 DIV의 크기를 감지합니다. 창 크기가 그대로여도 사이드바 접기나 부모 레이아웃 변경으로 호스트 크기가 바뀌면 그리드를 조정합니다.

설정 예제

다음 옵션은 gridProps 내부가 아닌 AUIGrid 서브 컴포넌트에 직접 지정합니다. columnLayout과 gridProps는 앞 절에서 정의한 값을 사용합니다.

<!-- 컨테이너 크기 변화에 따라 100ms 지연 후 조정합니다. -->

```
<AUIGrid
  ref="myGrid"
  resizeMode="container"
  :resizeDelayTime="100"
  :columnLayout="columnLayout"
  :gridProps="gridProps"
/>
```

기본값과 호출 시점

resizeMode의 기본값은 "window"입니다. 생략하면 기존 창 크기 이벤트 방식을 유지합니다. autoResize의 기본값은 true이며, :autoResize="false"로 지정하면 자동 감지를 등록하지 않습니다.

resizeDelayTime의 기본값은 300ms입니다. "container"에서는 마지막 알림 후 지정 시간만큼 기다리고 다음 애니메이션 프레임에 크기를 반영합니다. 0으로 설정해도 동기 실행하지 않습니다. 크기가 같거나 DOM이 분리되어 있거나 너비 또는 높이가 5px 이하이면 호출하지 않습니다.

resizeMode와 autoResize는 생성 시 지정하십시오. 실행 중 속성 변경만으로 감지 방식을 전환하지 않습니다. ResizeObserver가 없는 환경에서는 창 크기 이벤트로 대체합니다.

레이아웃과 생명주기

관찰 대상은 엔진 내부 셀이 아닌 호스트 DIV입니다. 앞 절처럼 width: "100%", height: 480을 사용하면 부모 너비에 맞추고 높이는 유지합니다. 높이도 부모에 맞추려면 호스트 CSS와 부모 높이를 지정하십시오. Flex/Grid에서는 필요에 따라 min-width: 0을 지정하고, 콘텐츠가 부모를 계속 늘리는 순환 배치를 피하십시오.

KeepAlive의 deactivated에서는 관찰과 예약 작업을 중단하고 activated에서 다시 연결합니다. 캐시된 그리드를 재생성하지 않습니다. 컴포넌트 해제 또는 래퍼의 destroy()에서는 관찰자와 타이머를 정리하고, create()로 다시 만들면 재연결합니다.

실제 resize()가 실행되면 편집 완료 등 엔진의 기존 리사이징 동작이 적용됩니다. 수동 resize() 사용법과 autoGridHeight의 데이터 기반 높이 정책은 유지됩니다.

- 감사합니다. -